

Building AI Agents with LangChain Tools

Using the Zain Jordan Customer 360 Database

Python Functions → LangChain Tools → AI Agent

Customer Care Use Case

Customer Care AI Agent

Profile
Plan
Risk
Billing
Usage

Complaints
Support
Action
Message

One Agent
Multiple
Tools

Formal class deck — no external branding

Where This Class Fits

A smooth path from database foundations to enterprise AI integration

Class 1
Database + Python Basics

Class 2
LangChain Tools + Agent

Class 3
SQL Agent

Class 4
RAG

Class 5+
Multi-Agent + MCP

Class 2 Focus

Convert the database functions built in Class 1 into tools an AI agent can call. The agent will use controlled access to customer data and produce a structured customer-care recommendation.

Class 2 Boundary

No MCP implementation yet. No RAG. No multi-agent system. No unrestricted database access. The goal is to master the tool-calling pattern first.

Learning Objectives

By the end of Class 2, participants should be able to explain and build a simple tool-using agent

Concepts

- Difference between chatbot and agent
- What a tool is in LangChain
- Why tool descriptions matter
- Why controlled tools are safer than open database access

Hands-On Skills

- Convert Python functions into LangChain tools
- Create a tool-using AI agent
- Ask customer-care questions using natural language
- Generate structured business recommendations

Final Class Output

A Customer Care AI Agent that can check profile, plan, churn risk, complaints, support history, billing, and usage for a selected customer ID.

Chatbot vs Agent

The key conceptual shift for participants

Normal Chatbot

- Responds directly
- Often text-only
- May guess when data is missing
- Does not decide what action to take
- Limited workflow control

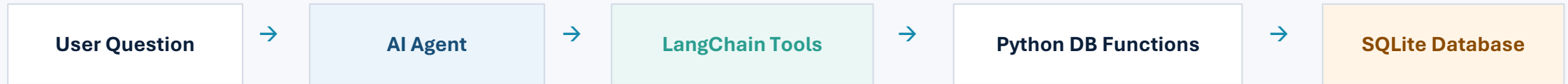
AI Agent

- Decides what information is needed
- Selects the right tool
- Calls database-backed functions
- Combines multiple tool results
- Produces an action-oriented answer

Simple message for learners: “A tool-using agent does not only answer — it checks, retrieves, compares, and recommends.”

Class 2 Architecture

A controlled, beginner-friendly agent design



Why This Design Works

The agent does not get unlimited database access. It can only call the tools we provide. This keeps the demo understandable, controlled, and suitable for enterprise training.

What the Agent Can Do

Retrieve customer data, inspect risk signals, summarize customer experience, identify billing concerns, and recommend the next best action.

Tools Used in Class 2

Each tool wraps a database function from Class 1

Tool	Purpose
get_customer_profile	Customer identity, city, segment, language, account status
get_customer_plan	Subscriptions, plan name, monthly fee, allowances
get_customer_churn_risk	Churn score, risk level, main reason, recommended action
get_customer_complaints	Recent complaint categories, severity, status, compensation
get_customer_support_interactions	Support history, channel, issue, priority, sentiment
get_customer_billing_summary	Invoices, payment status, overdue days
get_customer_usage_summary	Data sessions, total GB, cost, last activity

Main Use Case: Customer Care AI Agent

One agent, multiple customer-care tools

Business Prompt

Analyze customer 42. Check profile, plan, churn risk, recent complaints, support history, billing, and usage. Recommend the next best action and draft a short message.

Agent Task

Call the right tools, combine the results, avoid guessing, and produce a structured customer-care recommendation.

Profile

Plan

Risk

Complaints

Support

Billing

Usage

Structured Final Answer: Summary → Risk → Evidence → Recommended Action → Customer Message

Notebook Setup

What participants configure before running the agent

Required Packages

langchain
langchain-openai
langchain-community
pandas

Required Secrets

OPENAI_API_KEY
Stored in Colab Secrets
Notebook access enabled

Required File

zain_customer_360_ai_demo.db
Uploaded to Colab runtime
Auto-detected as DB_PATH

Trainer Check Before Demo

- Database file exists
- API key is loaded
- Tables are visible
- Customer ID 42 returns records
- Model name is available in your account

Step 1: Reuse Database Functions

The agent starts with normal Python functions

Class 1 Function Pattern

Input: customer_id
Action: run SQL query
Output: readable text or table result

Example Function

```
fetch_customer_profile(customer_id)
fetch_customer_plan(customer_id)
fetch_customer_churn_risk(customer_id)
```

Teaching Point

Before we create an AI tool, we test the normal function manually. This reduces debugging confusion for beginners.

Normal function first. Tool second. Agent third.

Step 2: Convert Functions into LangChain Tools

Tool names and descriptions guide the agent

Tool Name

A short, specific name such as `get_customer_churn_risk`. The model uses this to identify the tool.

Tool Description

A precise description of when the tool should be used and what it returns.

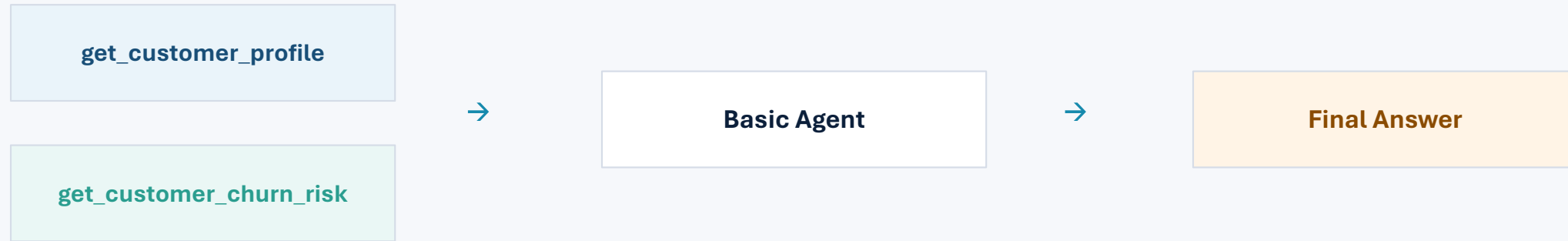
Tool Output

Clear text or structured data. For beginners, readable text is easier than complex objects.

```
@tool
def get_customer_churn_risk(customer_id: int) -> str:
    """Get churn score, risk level, reason, and recommended action."""
```

Step 3: Build the First Agent

Start with only two tools



First Demo Prompt

“Who is customer 42 and what is their churn risk?”

Expected Behavior

The agent calls profile and churn tools, then combines the results in a concise answer.

Step 4: Full Customer Care Agent

Expand the agent only after the first version works



Tools Available

- Profile
- Plan
- Churn risk
- Complaints
- Support interactions
- Billing summary
- Usage summary

System Instructions

- Use tools for customer facts
- Do not invent data
- Say when data is unavailable
- Use structured output
- Recommend a next action

Business Output

- Customer summary
- Plan summary
- Risk summary
- Experience summary
- Billing and usage
- Recommended action
- Customer-care message

Structured Output Format

A predictable answer format makes the agent useful for business teams

Section	What It Should Contain
Customer Summary	Name, city, segment, preferred language, account status
Plan Summary	Current plan, fee, data allowance, service type
Risk Summary	Churn score, risk level, main risk reason
Customer Experience	Complaints, support interactions, sentiment
Billing + Usage	Invoice status, overdue days, data usage
Recommended Action	Clear next step for support or retention team
Suggested Message	Short professional customer-facing response

Participant Exercises

Hands-on practice after the main demo

Exercise 1 Tool Selection

Match user questions to the correct tool: profile, plan, churn, complaints, support, billing, or usage.

Exercise 2 Customer Analysis

Analyze customer 10, 25, 42, and 100. Identify who needs urgent follow-up.

Exercise 3 Prompt Improvement

Change the system prompt to become a senior telecom retention specialist.

Optional Challenge

Create a new value-segment tool and use it to prioritize customers based on churn risk and revenue value.

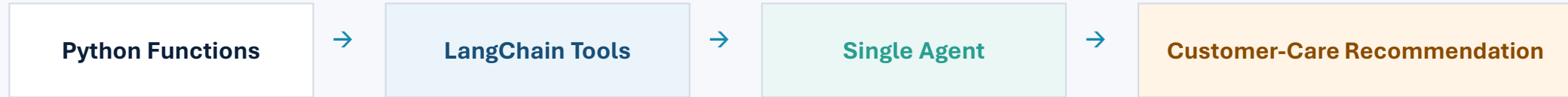
Common Errors and Quick Fixes

Practical troubleshooting for the live class

Issue	Likely Fix
OPENAI_API_KEY not found	Add key in Colab Secrets and enable notebook access
Database file not found	Upload .db file and confirm DB_PATH exists
Tool returns no records	Try another customer ID or explain that data is unavailable
Model unavailable	Change MODEL_NAME to an available OpenAI model
Agent gives vague answer	Improve tool descriptions and system prompt
Too much output	Limit rows and ask for structured summary

Class 2 Wrap-Up

The foundation for SQL agents, RAG, multi-agent workflows, and MCP



What Participants Built

A tool-using customer-care agent that can retrieve customer data, assess risk signals, and generate a business-friendly recommendation.

Next Class

Natural Language SQL Agent: ask broader telecom business questions directly from the database.

Final message: master tools first, then move to SQL agents, RAG, multi-agent systems, and MCP.